

Strathmore University

@iLabAfrica Centre

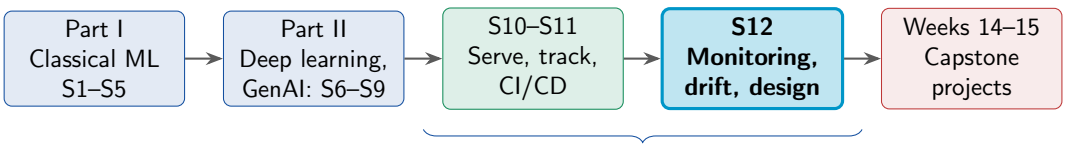
Session 12: Model Monitoring, Drift & ML System Design

Javier Serrano

Applied Machine Learning

Master in Data Science & Analytics

Where this session sits in the course



Takeaway: Sessions 10 and 11 put the model into production. Today: how do we know it is still right next month?

Learning outcomes

By the end of this session you will be able to:

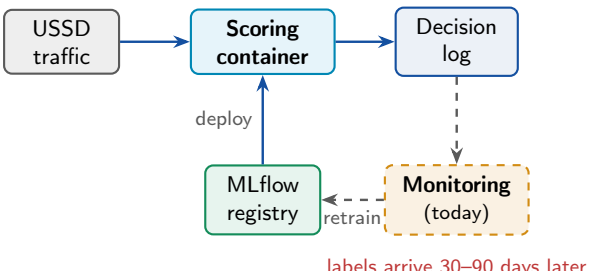
1. Explain why deployed models degrade silently.
2. Classify a shift as covariate, label or concept drift.
3. Detect drift with KS, Wasserstein, PSI and a domain classifier.
4. Automate drift checks with Evidently AI and a CI gate.
5. Design ML systems that scale, degrade gracefully and stay fair.

The running case is now live

The Nairobi thin-file scorer after Sessions 10 and 11

Case study: Nairobi thin-file scorer

- Serves USSD loan requests in under 2 s.
- Default rate $\pi \approx 4\%$ at training time.
- Threshold $t^\star \approx 0.074$ from the KES 10,000 / 800 cost matrix.
- Promoted as `@champion` in MLflow.



Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

ML system design principles

Responsible ML and governance

Wrap-up

ML systems fail silently

A crashed service pages someone; a wrong model returns 200 OK

Software failure: loud

HTTP 500, timeouts, crash loops
dashboards turn red at once
on-call engineer is paged
fixed in hours

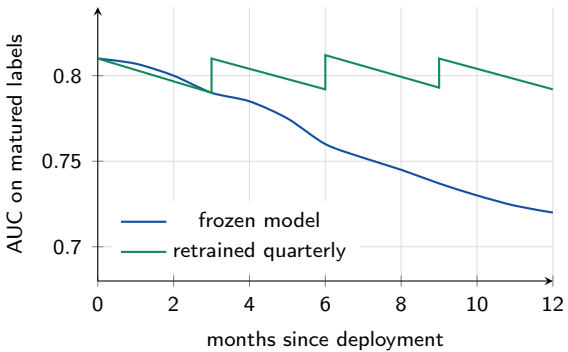
ML failure: silent

HTTP 200, valid JSON, fast
every health check passes
decisions are quietly worse
found when defaults arrive

- Health checks from Session 10 prove the service is *alive*, not that it is *right*.
- Monitoring must watch the data and the predictions, not only the server.

Performance decays after deployment

Model quality is a function of time, not a constant

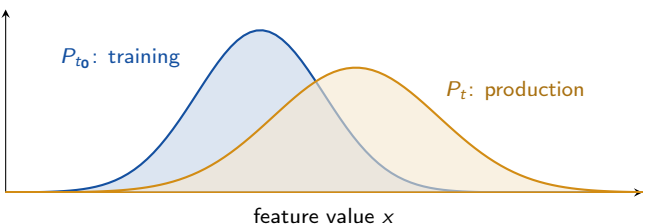


- The offline test AUC describes the world *at training time*.
- Customers, products and the economy keep moving.
- Retraining restores quality, but only if you know *when*.

The assumption that breaks

Supervised learning assumes train and production data come from one distribution

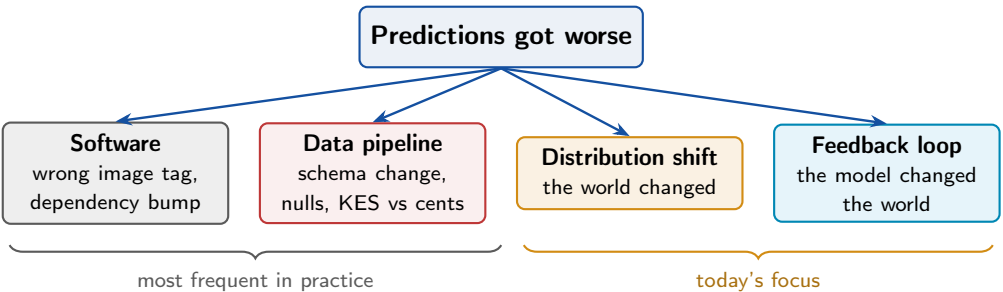
$$\mathcal{D}_{\text{train}} \sim P_{t_0}(X, Y) \qquad \text{but} \qquad \mathcal{D}_{\text{prod}} \sim P_t(X, Y), \quad P_t \neq P_{t_0}$$



Takeaway: Empirical risk minimisation guarantees nothing once P_t moves away from P_{t_0}.

Not every degradation is drift

Check the cheap explanations first [Huyen, 2022]



Pitfall: blaming drift for a bug

An upstream team changes `avg_monthly_inflow` from KES to cents. Every statistical detector fires, yet retraining is the wrong fix.

Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

ML system design principles

Responsible ML and governance

Wrap-up

Which part of the joint distribution moved?

Two factorisations, three kinds of drift [Gama et al., 2014]

$$P(X, Y) = \underbrace{P(Y \mid X)}_{\text{concept}} \underbrace{P(X)}_{\text{inputs}} = \underbrace{P(X \mid Y)}_{\text{class profile}} \underbrace{P(Y)}_{\text{prior}}$$

Covariate shift

$P(X)$ changes
$P(Y \mid X)$ fixed

who applies changes

Label shift

$P(Y)$ changes
$P(X \mid Y)$ fixed

how many default changes

Concept drift

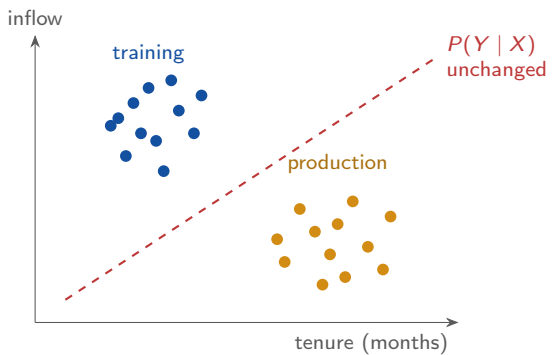
$P(Y \mid X)$ changes

what the signals mean changes

- Each type has a different detector and a different fix.

Covariate shift

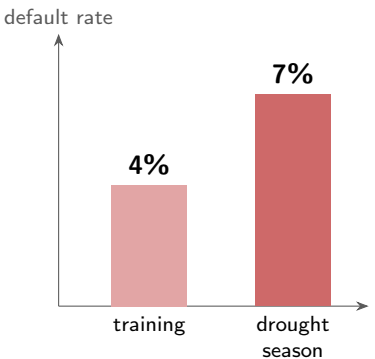
New kinds of applicants, same relationship between signals and default



- The rule is still right, but the model now works in regions it rarely saw.
- **Example**: a partnership brings younger, newer users from western Kenya.
- **Detectable without labels**: compare input distributions.
- **Fix**: often none; retrain if error grows in the new region.

Label shift (prior shift)

The default rate moves; each class still looks the same



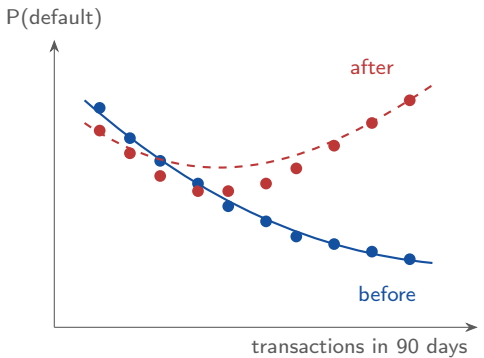
Bayes' rule gives the correction without retraining [Saerens et al., 2002]:

$$\tilde{p}(y \mid x) = \frac{w_y \, p(y \mid x)}{\sum_{y'} w_{y'} \, p(y' \mid x)}, \qquad w_y = \frac{\pi'_y}{\pi_y}$$

- With $\pi : 4\% \to 7\%$, the decline threshold on the *old* scores drops from 0.074 to about 0.042.
- **Fix**: re-estimate π' and recalibrate; no new features needed.

Concept drift

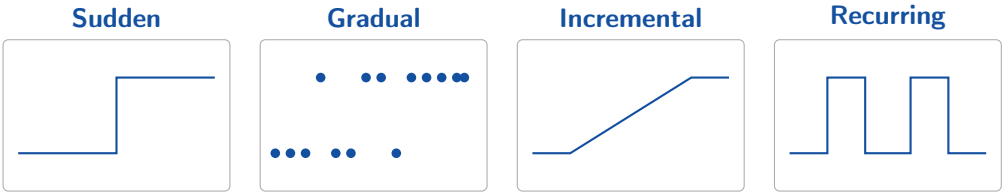
The same signals now mean something different



- High activity used to signal a healthy business.
- After a new overdraft product, very high activity can signal a borrower juggling several loans.
- **Invisible in** $P(X)$: needs labels, or strong proxies.
- **Fix**: retrain on recent labelled data.

How drift unfolds in time

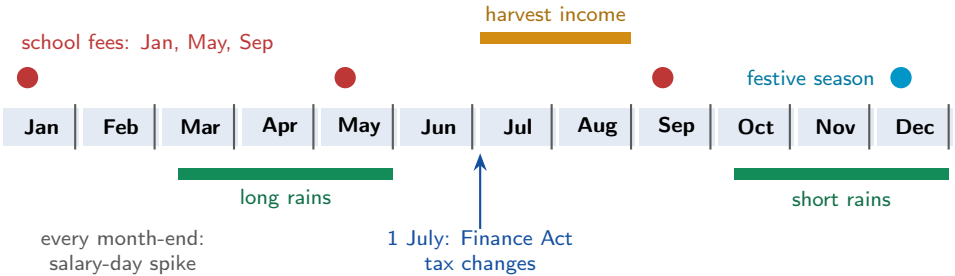
The shape of the change decides the detection window



- Short windows catch sudden drift but raise false alarms on recurring patterns.
- Recurring drift is *expected*: compare with the same period last year.

A Kenyan drift calendar

Know the rhythms before you alarm on them



Kenyan context

Seasonal features (school-fee month, rains) belong in the model. Unexpected events, such as a general election or a new mobile-money levy, are the drift to detect.

What can we see without labels?

Most monitoring happens before the ground truth arrives

Drift type	Watch the inputs	Watch the scores	Needs labels?
Covariate	yes	yes	no
Label	partly	yes	to confirm
Concept	no	rarely	yes

Takeaway: Label-free monitoring is an *early warning*. Performance on matured labels is the *verdict*.

Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

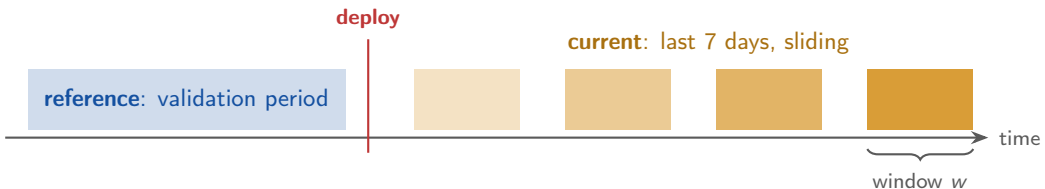
ML system design principles

Responsible ML and governance

Wrap-up

Reference and current windows

Every drift check compares two samples



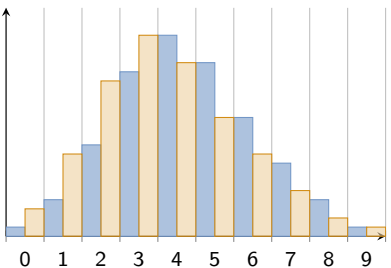
- **Reference**: a period where the model was known to be good (usually the validation set).
- **Window size**: small w reacts fast but is noisy; large w is stable but slow.
- Use a window that covers a full weekly cycle, so weekends do not look like drift.

Drift detection as a two-sample test

Did both windows come from the same distribution?

$H_0 : P_{\text{ref}} = P_{\text{cur}}$ $\quad$ $H_1 : P_{\text{ref}} \neq P_{\text{cur}}$

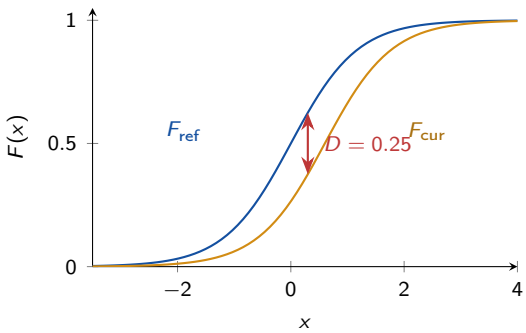
- A statistic $d(\hat{P}_{\text{ref}}, \hat{P}_{\text{cur}})$ measures the gap.
- Two families:
  - **tests**: KS, χ^2, give p-values;
  - **distances**: Wasserstein, PSI, JS, give effect sizes.
- Run per feature, on the model score, and jointly.



reference vs **current** window

The Kolmogorov–Smirnov statistic

The largest vertical gap between two empirical CDFs

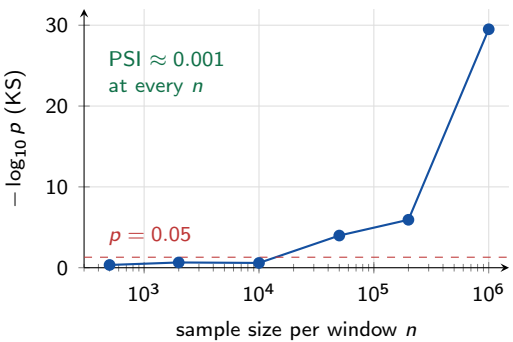


$$D = \sup_x \left| \hat{F}_{\text{ref}}(x) - \hat{F}_{\text{cur}}(x) \right|$$

- Distribution-free, for continuous features.
- Reject H_0 when D exceeds a critical value that shrinks like $\sqrt{(n+m)/(nm)}$.
- Sensitive near the median, less in the tails.

The large-sample trap

With enough data, every difference is "significant"

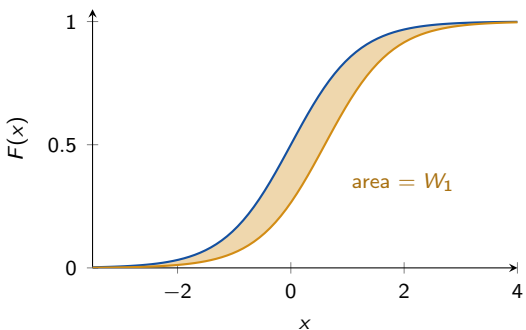


- Same tiny shift (0.02 standard deviations) at every n.
- The p-value collapses; the business impact does not change.
- A scorer handling 50,000 requests a day will alarm every day.

Takeaway: Alert on **effect size**, not on p-values.

Wasserstein distance

How much probability mass must move, and how far

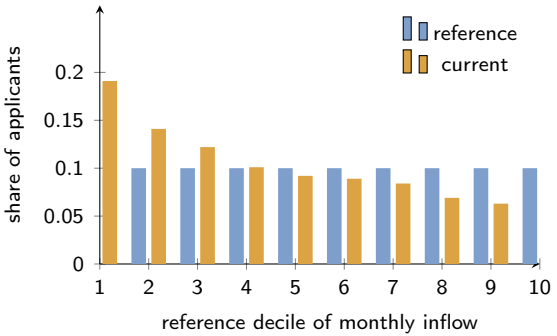


$$W_1 = \int_{-\infty}^{\infty} \left| F_{\text{ref}}(x) - F_{\text{cur}}(x) \right| dx$$

- In the units of the feature: easy to explain.
- Evidently divides by σ_{ref} and flags drift when the result exceeds 0.1.
- Stable at large n: it converges to the true gap, not to zero.

Population Stability Index (PSI)

The credit industry's standard drift measure [Siddiqi, 2006]



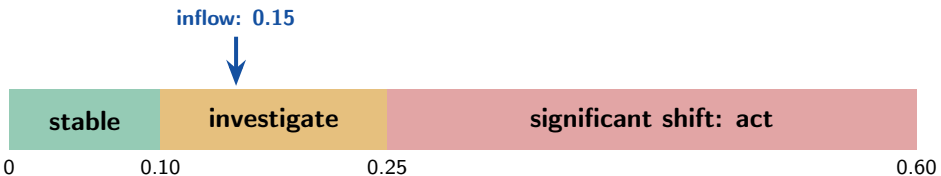
Bin the reference into B quantile bins with shares p_b; let q_b be the current shares:

$$\mathrm{PSI} = \sum_{b=1}^{B} (q_b - p_b) \ln \frac{q_b}{p_b}$$

- A symmetrised KL divergence.
- Here: $\mathrm{PSI} = 0.15$.

Reading a PSI value

Traffic lights used by credit-risk teams



- These cut-offs are conventions from scorecard practice, not theorems: tune them on your own history.
- PSI depends on the binning. Fix the bin edges from the reference and store them with the model.
- Empty bins make $\ln(q_b/p_b)$ explode: floor shares at a small ε.

PSI and KS in a few lines

Bin edges come from the reference only

```python
import numpy as np
from scipy import stats

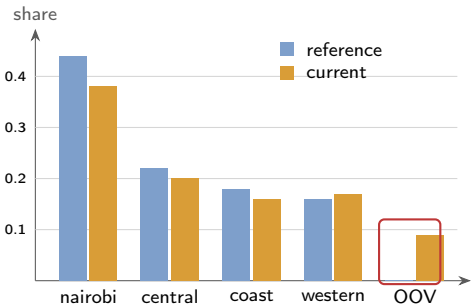
def psi(ref, cur, bins=10, eps=1e-6):
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1))    # reference deciles
    edges[0], edges[-1] = -np.inf, np.inf                     # catch out-of-range values
    p = np.histogram(ref, edges)[0] / len(ref)
    q = np.histogram(cur, edges)[0] / len(cur)
    p, q = np.clip(p, eps, None), np.clip(q, eps, None)       # no log(0)
    return float(np.sum((q - p) * np.log(q / p)))

col = "avg_monthly_inflow_kes"
print("PSI :", psi(ref_df[col], cur_df[col]))                                  # 0.15  [SIMULATED]
print("KS  :", stats.ks_2samp(ref_df[col], cur_df[col]).statistic)             # 0.16
print("W1  :", stats.wasserstein_distance(ref_df[col], cur_df[col]) / ref_df[col].std())
```

Takeaway: In production, persist edges next to `scorer.keras`: they are part of the monitoring contract.

Categorical features

Chi-squared and Jensen–Shannon on category shares



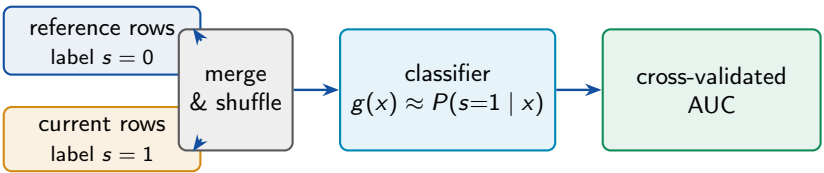
With $M = \frac{1}{2}(P + Q)$:

$$\mathrm{JS} = \tfrac{1}{2}\,\mathrm{KL}(P \,\|\, M) + \tfrac{1}{2}\,\mathrm{KL}(Q \,\|\, M)$$

- Bounded and symmetric; defined even when a category is new.
- **OOV share** is a free signal: Session 10's `StringLookup` routes unseen codes to one bucket.
- 9% OOV usually means an upstream change.

Multivariate drift: the domain classifier

Can a model tell yesterday's applicants from today's? [Rabanser et al., 2019]



- AUC ≈ 0.5: the windows are indistinguishable, so no joint drift.
- AUC $\gg 0.5$: drift somewhere, including in *interactions* that per-feature tests miss.
- The classifier's feature importances tell you *where* to look.

Domain classifier in code

Reuse the gradient-boosting tools from Session 4

```
import numpy as np, pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import roc_auc_score

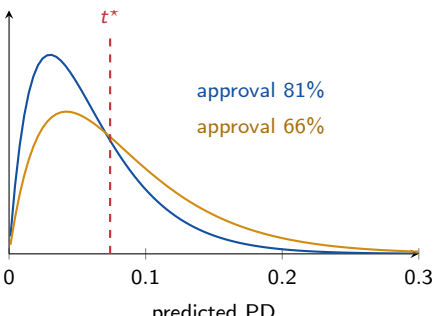
X = pd.concat([ref_df[FEATURES], cur_df[FEATURES]], ignore_index=True)
s = np.r_[np.zeros(len(ref_df)), np.ones(len(cur_df))]          # which window?

clf = HistGradientBoostingClassifier(max_iter=200, categorical_features="from_dtype")
p = cross_val_predict(clf, X, s, cv=5, method="predict_proba")[:, 1]  # out-of-fold scores
auc = roc_auc_score(s, p)
print(f"domain AUC = {auc:.3f}")        # 0.50: no drift; 0.62: investigate  [PLACEHOLDER]
```

Takeaway: Use out-of-fold predictions (in-sample AUC is optimistic), and drop timestamps and IDs first, or the classifier learns the calendar.

Prediction drift: watch the model's own output

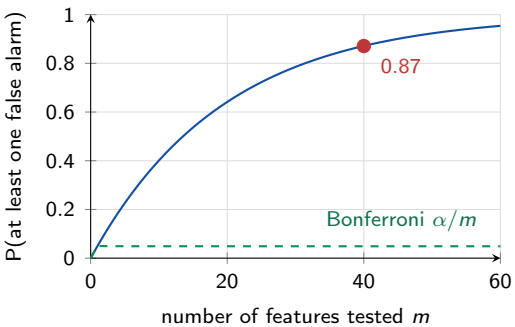
A one-dimensional summary of everything the model sees



- Track the PD histogram (PSI on scores) and the approval rate.
- Available at request time: no labels needed.
- A falling approval rate is the first thing the business will notice. Notice it first.

Many features, many tests

False alarms multiply with the number of monitored columns



With m independent tests at level α:

$$\Pr(\text{false alarm}) = 1 - (1 - \alpha)^m$$

- 40 features, $\alpha = 0.05$: an alarm on 87% of *stable* days.
- Remedies: Bonferroni, a **drift share** rule, or weighting by feature importance.

Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

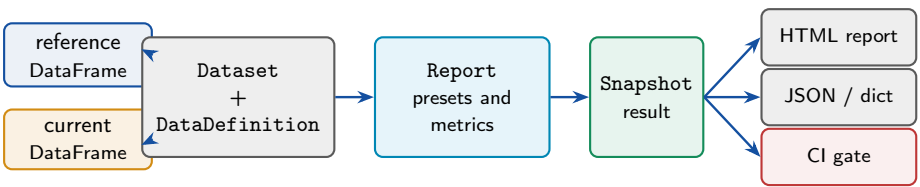
ML system design principles

Responsible ML and governance

Wrap-up

What Evidently does

Two DataFrames in, drift evidence out [Evidently AI docs, v0.7]



- Chooses a sensible test per column, runs it, and explains the result visually.
- One snapshot feeds a human (HTML) and a machine (a dict for a gate).

Running a drift report

Evidently $\geq$ 0.7 API

```python
from evidently import Report, Dataset, DataDefinition
from evidently.presets import DataDriftPreset

schema = DataDefinition(
    numerical_columns=["tenure_months", "avg_monthly_inflow_kes",
                       "txn_count_90d", "ontime_repayments_12m", "pd"],
    categorical_columns=["region"])

ref = Dataset.from_pandas(ref_df, data_definition=schema)     # validation period
cur = Dataset.from_pandas(cur_df, data_definition=schema)     # last 7 days of the decision log

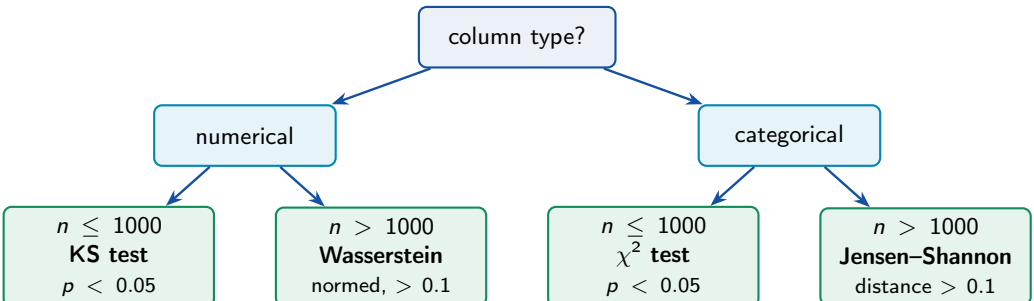
report = Report([DataDriftPreset(drift_share=0.3)])           # dataset drift if >= 30% of columns drift
snapshot = report.run(current_data=cur, reference_data=ref)
snapshot.save_html("reports/drift_2026-10-19.html")           # for humans
```

Pitfall: two APIs in the wild

Most tutorials use the pre-0.7 API (`evidently.report`), now under `evidently.legacy`. Pin the version.

What the preset decides for you

Default drift method per column

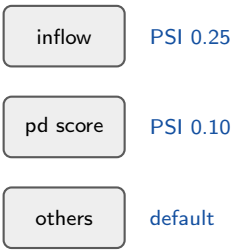


- Notice the pattern from the large-sample trap: tests for small n, distances for large n.
- Binary categoricals with small n use a proportion Z-test.

Override the defaults where you know better

PSI on the columns your risk team already tracks

```
preset = DataDriftPreset(
    drift_share=0.3,
    per_column_method={
        "avg_monthly_inflow_kes": "psi",
        "pd":                     "psi",
    },
    per_column_threshold={
        "avg_monthly_inflow_kes": 0.25,
        "pd":                     0.10,   # stricter on
     scores
    },
)
snapshot = Report([preset]).run(current_data=cur,
                                reference_data=ref)
```



- Stricter on the score: it summarises all inputs.
- Record every override in the model card.

From report to gate

check_drift.py returns an exit code a pipeline can act on

```
import json, sys

metrics = snapshot.dict()["metrics"]
summary = next(m for m in metrics if m["metric_name"].startswith("DriftedColumnsCount"))
share = summary["value"]["share"]                    # fraction of drifted columns
drifted = [m["config"]["column"] for m in metrics
           if m["metric_name"].startswith("ValueDrift")
           and m["value"] > m["config"]["threshold"]]  # distance-based methods

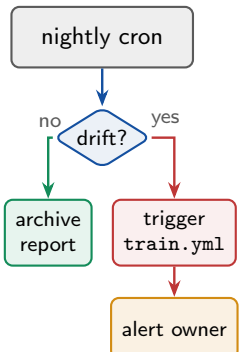
json.dump({"share": share, "drifted": drifted}, open("reports/drift.json", "w"))
print(f"drift share = {share:.2f}; columns: {drifted}")
sys.exit(1 if share >= 0.3 else 0)                   # non-zero exit = drift detected
```

Takeaway: The HTML is for people; the exit code is for automation. Ship both from the same run.

Wiring the gate into the pipeline

A nightly job that can trigger Session 11's retraining workflow

```yaml
name: drift-check
on:
  schedule:
    - cron: "0 2 * * *"       # 05:00 EAT
jobs:
  drift:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - run: python monitoring/export_window.py
      - id: gate
        run: python monitoring/check_drift.py
      - if: failure()
        run: gh workflow run train.yml
```



- Retraining still passes the Session 11 promotion gates: drift triggers a *candidate*, never a direct deploy.

Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

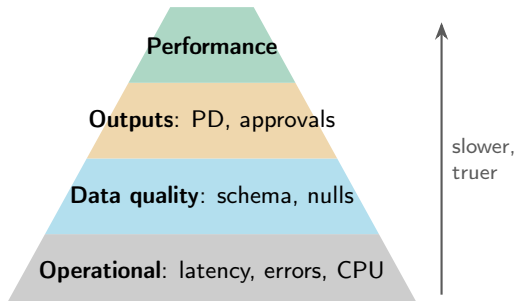
ML system design principles

Responsible ML and governance

Wrap-up

Four layers of monitoring

From the cheapest, fastest signal to the slowest, most decisive one

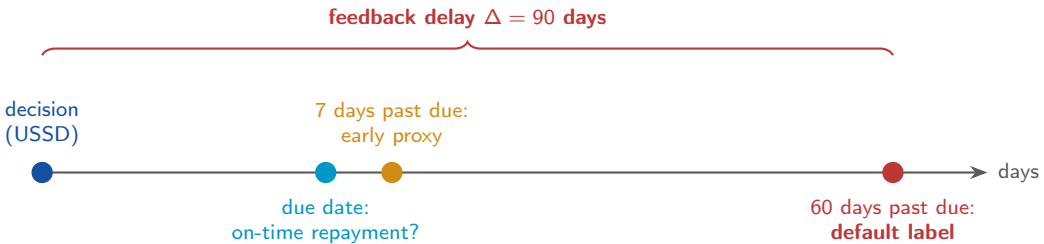


- **Seconds**: operational metrics.
- **Minutes**: data-quality checks on each batch.
- **Days**: score and input drift.
- **Months**: AUC and cost on matured labels.

Takeaway: Monitor every layer: each catches failures the others miss.

The delayed-label problem

In lending, the truth arrives months after the decision



- A concept drift that starts today appears in AUC three months from now.
- **Proxies** (first-payment behaviour) arrive sooner but are noisier.
- Until then, rely on input, score and data-quality monitoring.

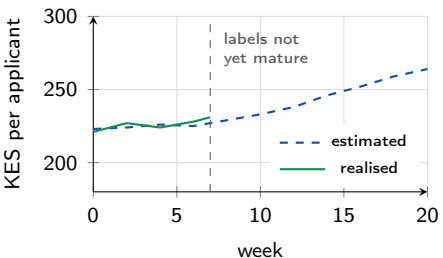
Estimating performance before labels arrive

A calibrated model predicts its own expected cost

If p_i is calibrated and only covariate shift occurred, the expected cost of this week's decisions is

$$\widehat{\text{Cost}} = \frac{1}{n}\sum_{i=1}^{n}\Big[a_i\, p_i\, C_{FN} + (1-a_i)(1-p_i)\, C_{FP}\Big]$$

- $a_i = \mathbb{1}[p_i < t^\star]$: approved. No labels needed.



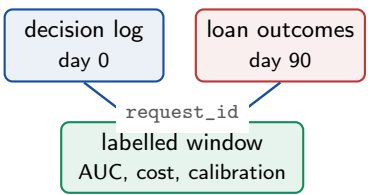
Pitfall: blind to concept drift

If $P(Y \mid X)$ moved, calibration breaks and the estimate is wrong.

Log what you will need to join later

The decision log is the raw material of all monitoring

```
record = {
  "request_id": rid,            # join key
  "ts": now_utc_iso(),
  "model_version": meta["version"],
  "threshold": meta["threshold"],
  "features": feats,            # no MSISDN
  "pd": pd_,
  "decision": decision,
  "latency_ms": round(1e3 * elapsed, 1),
}
decision_log.write(json.dumps(record))
```



Kenyan context

DPA 2019 purpose limitation: log what monitoring needs, set a retention period, and keep the MSISDN hash salt out of the log store.

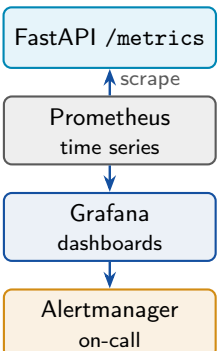
Operational metrics from the API

Expose them once; Prometheus scrapes, Grafana draws, Alertmanager pages

```python
from prometheus_client import (Counter,
        Histogram, make_asgi_app)

LATENCY = Histogram("score_latency_seconds",
        "Scoring latency",
        buckets=(.01, .05, .1, .15, .3, 1, 2))
DECISIONS = Counter("decisions_total",
        "Decisions", ["decision", "version"])
app.mount("/metrics", make_asgi_app())

@app.post("/v1/score", response_model=ScoreResponse)
def score(a: Applicant) -> ScoreResponse:
    with LATENCY.time():
        r = _score(a)            # Session 10 logic
    DECISIONS.labels(r.decision, r.model_version).inc()
    return r
```



Alerting without alert fatigue

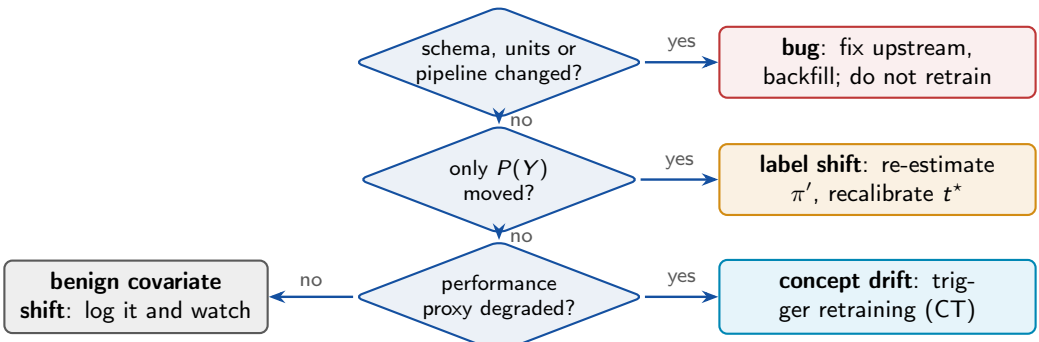
An alert nobody acts on trains people to ignore alerts

Latency	**Errors**	**Data quality**	**Drift**	**Business**
$p_{99} > 150$ ms for 10 min	5xx $> 1\%$ for 5 min	nulls or OOV $> 5\%$ per batch	score PSI > 0.1 or share ≥ 0.3	approval rate moves > 5 pp
page on-call	page on-call	ticket, data team	ticket, ML owner	risk committee

- Every alert has an **owner**, a **severity** and a **runbook**.
- Page humans only for what needs action tonight; drift usually goes to a ticket.

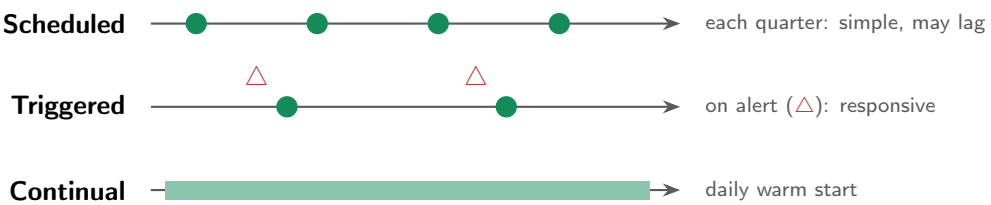
Triage: from alert to action

Diagnose before you retrain



When to retrain

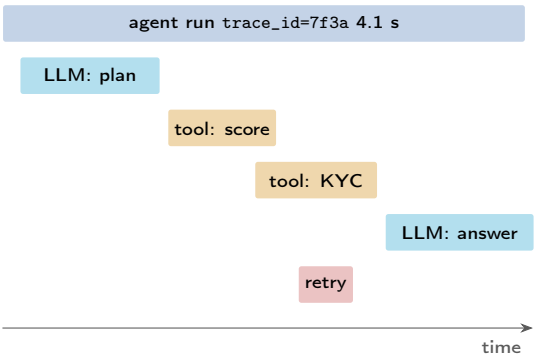
Three strategies, one trade-off: staleness against cost and risk



- Start **scheduled** + **triggered**: the Session 11 pipeline already supports both.
- Continual learning needs fast labels; lending labels are slow, so it rarely pays here.
- Every retrain is a new version that must pass the promotion gates.

Monitoring LLM agents

Session 10's underwriting agent needs traces, not just logs



- **Per run**: latency, tokens, cost in KES, number of steps.
- **Per tool call**: success rate, schema errors, retries.
- **Quality**: groundedness and policy checks, scored by evaluators on a sample.
- **Drift**: prompt topics and languages (English, Kiswahili, Sheng) change too.

Takeaway: The same principles apply: a reference window, a distance, a threshold, an owner.

Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

ML system design principles

Responsible ML and governance

Wrap-up

Four properties of a good ML system

Huyen's requirements, mapped to what Sessions 10–12 built [Huyen, 2022]

Reliability: correct even when inputs are bad or services fail.
S10: contracts, fail closed. S12: monitoring.

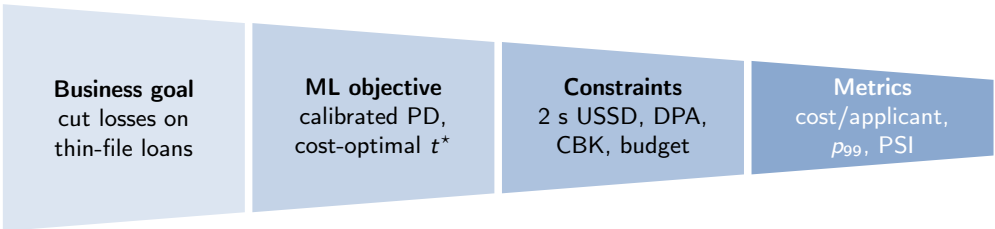
Scalability: handles salary-day peaks and new products.
S10: workers, Little's law. S12: autoscaling.

Maintainability: others can read, test and change it.
S11: MLflow lineage, tests, CI.

Adaptability: absorbs drift and new data quickly.
S11: CT pipeline. S12: drift gates.

Design starts from requirements, not models

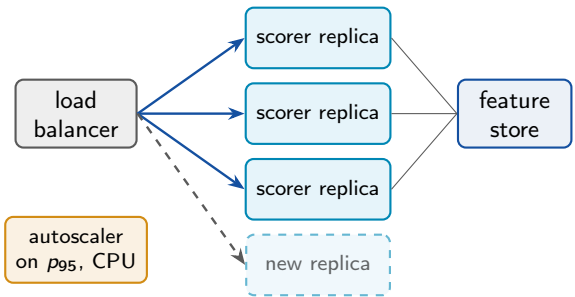
Write the constraints down before choosing an architecture



- Each monitored metric today traces back to a requirement on this slide.
- If a metric has no requirement behind it, nobody will act when it alarms.

Scaling out: stateless replicas

Keep the scorer stateless and let an autoscaler add copies



Recall from Session 10:

$$c = \left\lceil \frac{\lambda \bar{S}}{\rho_{\max}} \right\rceil$$

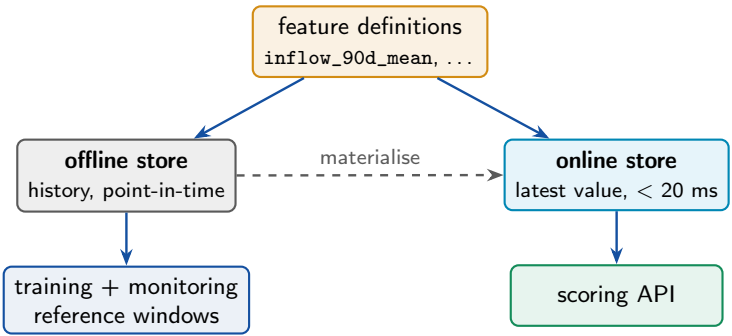
Salary day (*placeholder*): λ rises from 120 to 400 req/s with $\bar{S} = 25$ ms and $\rho_{\max} = 0.6$:

$$c : 5 \rightarrow 17 \text{ replicas.}$$

- State lives in the feature store and the log, never in a replica.

One feature definition, two stores

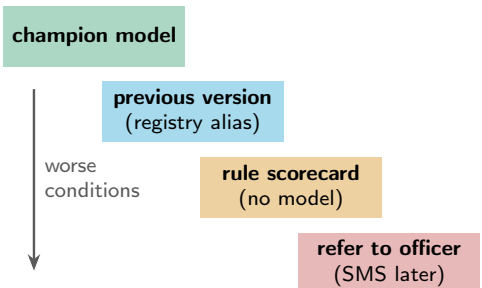
A feature store removes training–serving skew at the source



- **Point-in-time joins** prevent label leakage when building training sets.
- Drift is computed on exactly the values the model received.

Graceful degradation

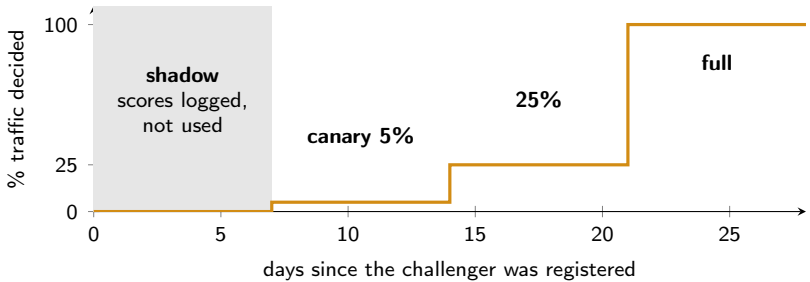
Design the fallback ladder before you need it



- Model fails validation or feature store times out: step down one rung.
- The last rung **fails closed**: never an automatic approval.
- Each rung is tested in CI, like any other code path.

Safe rollout: shadow, canary, full

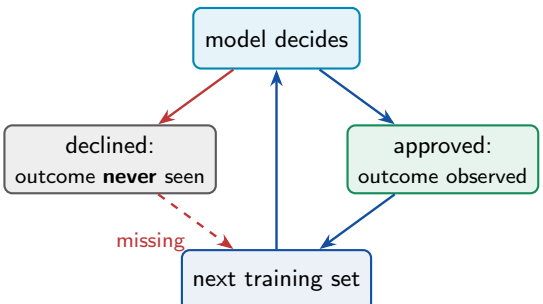
Limit the blast radius of every new version



- Compare champion and challenger on the *same* requests: PD distribution, approvals, latency.
- Rollback is one registry operation: point `@champion` back to the previous version.

Feedback loops and selective labels

The model decides which outcomes you will ever see [Lakkaraju et al., 2017]



We only ever learn

$$P(Y \mid X,\ \text{approved}) \neq P(Y \mid X)$$

- Past decisions shape future data: a drift you cause yourself.
- Remedies: a small, randomised **exploration** budget; reject inference; human-review samples of declines.

Designing for Kenyan infrastructure

Constraints are design inputs, not excuses

Connectivity
USSD sessions time out; retries must be idempotent

Data residency
DPA 2019 transfer rules; keep logs in-country

Cost
CPU-first models; GPUs only where they pay

Power & links
resumable jobs, local registry mirror

Kenyan context

A monitoring stack you cannot afford to run is a stack you will switch off. Evidently, Prometheus and Grafana run on one modest self-hosted server.

Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

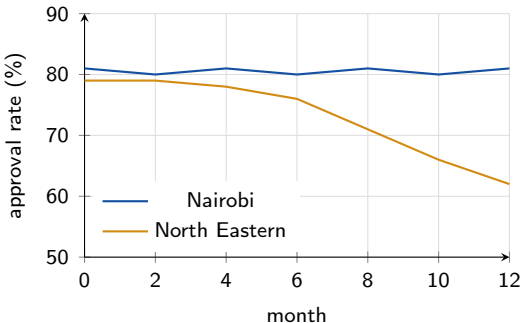
ML system design principles

Responsible ML and governance

Wrap-up

Fairness can drift too

A model that was fair at launch need not stay fair



- A drought hits one region's inflows first.
- The model is still accurate on average, but decisions diverge by group.
- Aggregate monitoring hides this: monitor **slices**.

Two fairness criteria

Choose one explicitly and document the choice [Hardt et al., 2016]

Demographic parity: equal approval rates

$$P(\hat{Y}{=}1 \mid a) = P(\hat{Y}{=}1 \mid b)$$

Equal opportunity: equal approval among those who repay

$$P(\hat{Y}{=}1 \mid Y{=}0, a) = P(\hat{Y}{=}1 \mid Y{=}0, b)$$

($\hat{Y} = 1$: approve; $Y = 0$: repays; a, b: groups)



- Equal opportunity needs labels, so it inherits the 90-day delay.
- When base rates differ, calibration and equal error rates cannot all hold [Chouldechova, 2017].

Slice monitoring in code

Per-group rates with a minimum-sample guard

```python
import pandas as pd

def slice_report(log: pd.DataFrame, by: str, min_n: int = 200) -> pd.DataFrame:
    g = log.groupby(by)
    out = pd.DataFrame({
        "n":        g.size(),
        "approval": g["decision"].apply(lambda d: (d == "approve").mean()),
        "mean_pd":  g["pd"].mean(),
    })
    out["gap_pp"] = 100 * (out["approval"] - out["approval"].max())   # vs best-treated group
    return out[out["n"] >= min_n]            # small slices: report n, do not alarm

weekly = slice_report(decision_log_df, by="region")
alerts = weekly[weekly["gap_pp"] < -10]      # [PLACEHOLDER] policy threshold
```

Pitfall: tiny slices

A slice with 30 applicants swings 10 points by chance. Report its size and confidence interval before raising an alarm.

Governance: obligations become artefacts

Compliance as a by-product of a well-built pipeline

Obligation	Artefact the pipeline already emits
DPA 2019 s.25: data minimisation	hashed MSISDN; no raw identifiers in the decision log
DPA 2019 s.35: automated decisions	`refer` path, human review queue, reason codes
CBK DCP Regs 2022: complaints	`request_id` audit trail, model version per decision
Model risk management	MLflow lineage, promotion gates, archived drift reports
Fairness commitments	weekly slice report, signed off by the risk owner

Takeaway: If an auditor's question needs a manual investigation, the pipeline is missing an artefact.

Model cards, regenerated on every promotion

Documentation that cannot go stale [Mitchell et al., 2019]

thinfile-scorer 1.4.0

Intended use: thin-file USSD loans

Data: snapshot + hash

Metrics: AUC, cost, calibration

Slices: region, age band

Monitoring: PSI and drift thresholds



- Generated by the Session 11 pipeline from MLflow tags and evaluation outputs.
- States the intended use and the *out-of-scope* uses.
- Records the monitoring plan: which metrics, which thresholds, which owner.
- Stored with the model version in the registry.

Roadmap

Why deployed models degrade

A taxonomy of drift

Detecting drift statistically

Drift detection with Evidently AI

Monitoring in production

ML system design principles

Responsible ML and governance

Wrap-up

Pitfalls to avoid

Each of these has misled a real monitoring team

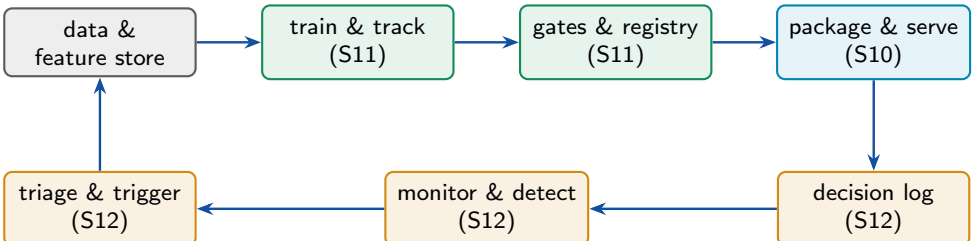
Five monitoring pitfalls

1. Alerting on p-values with huge windows.
2. Retraining to "fix" an upstream data bug.
3. Monitoring only aggregates, never slices.
4. No reference window stored with the model.
5. Alerts without an owner or a runbook.



The complete production loop

Sessions 10, 11 and 12 in one picture



- Detect drift early without labels; confirm it with matured labels.
- Diagnose before acting: bug, label shift, concept drift, or benign.
- Design for scale, graceful failure and fairness from the first diagram.

Bridge to the capstone

"Show me your drift alert" is a question at the final presentation

- [x] a stored reference window and bin edges
- [x] a drift report (Evidently) on a realistic shifted window
- [x] an automated gate with an exit code
- [x] one slice metric relevant to your domain
- [x] a one-page runbook: owner, thresholds, actions

Takeaway: A capstone system that cannot tell you when it is wrong is not production-ready.

Looking ahead: Weeks 14–15, capstone presentations: every team demonstrates its drift alert.

Summary

- Deployed models fail silently: monitor the data and the predictions, not only the server.
- Covariate, label and concept drift differ in what moves, how you see it and how you fix it.
- Alert on effect sizes (Wasserstein, PSI, JS, domain AUC), not on p-values from huge windows.
- Evidently turns two windows into an HTML report for people and an exit code for pipelines.
- Diagnose before retraining; design for scale, graceful failure, safe rollout and fairness.

References

- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly. Ch. 8, 9, 11.
- Géron, A. (2019). *Hands-On ML*, 2nd ed. Ch. 2, "Launch, monitor, and maintain your system" (same section in the 3rd ed.).
- Gama, J. et al. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*.
- Rabanser, S., Günnemann, S. & Lipton, Z. (2019). Failing loudly: detecting dataset shift. NeurIPS.
- Lipton, Z., Wang, Y.-X. & Smola, A. (2018). Detecting and correcting for label shift with black box predictors. ICML.
- Mitchell, M. et al. (2019). Model cards for model reporting. FAccT.
- Evidently AI documentation (presets, drift methods); Prometheus client for Python.